PRODUCTION READINESS AUDIT

DeepMindQ Production Foundation v1.0

Starting Point

Frozen baseline snapshot captured before implementation of the Production Foundation v1.0 recovery plan. This document records the exact state of the DeepMindQ platform: code metrics, test results, runtime configuration, database state, and production readiness scores. No code modifications were made during the creation of this checkpoint.

Commit: f2a971a | Branch: main | Date: 2026-08-01

Table of Contents

1. Code Baseline

The code baseline captures the essential structural metrics of the DeepMindQ platform at the moment this checkpoint was recorded. These numbers represent the frozen state before any v1.0 remediation work begins. All values were obtained through direct filesystem inspection and git operations on commit f2a971af5b15050c44a180184ea972477cb61753. No uncommitted changes exist in the working tree. The platform consists of 626 TypeScript/TSX source files totaling 182,513 lines of code, 223 API route handlers, 77 screen components, 96 Prisma models, and a 2,909-line schema definition.

1.1 Repository State

Metric	Value
Git Commit Hash	f2a971af5b15050c44a180184ea972477cb61753
Branch	main
Uncommitted Changes	None (clean working tree)
Package Version	0.2.0
Next.js Version	16.1.1
Prisma Client	6.19.3
TypeScript	5.x
Vitest	4.1.10

1.2 Code Metrics

Metric	Value
Total TypeScript Files	626
Total Lines of Code (src/)	182,513
API Route Handlers	223
Screen Components	77
Prisma Models	96
Prisma Enums	20
Prisma Schema Lines	2,909
Proxy Middleware Lines	221
Database Size	2.0 MB (SQLite)

1.3 API Route Authentication Coverage

Of the 223 total API route handlers in the codebase, only **1 route** imports any form of authentication middleware (`withApiMiddleware`). The proxy middleware at `src/proxy.ts:95` checks only for the `dmq_session` cookie presence, not its validity against the database. This

means that **222 of 223 API routes** are effectively unprotected. The `checkApiAuth()` and `requireAdminRole()` functions exist in `src/lib/api-auth.ts` and are fully exported, but they are not imported by any route handler. Additionally, 105 routes use the `apiSuccess` or `utilitySuccess` response envelope patterns, while the remaining 118 routes return raw JSON without consistent envelope formatting.

Category	Count	Details
Total API Routes	223	All route.ts files under src/app/api/
Protected (auth middleware)	1	`engines/brief/route.ts uses withApiMiddleware`
Unprotected	222	`No checkApiAuth, requireAdminRole, or withApiMiddleware import`
Envelope-Compliant	105	`Use apiSuccess or utilitySuccess wrappers`
Non-Envelope	118	`Return raw JSON without standardized wrapper`

2. Test Baseline

The test baseline records the exact state of the test suite at this checkpoint. Tests were executed using `npx vitest run` with no filters, capturing the full 48-file suite of 1,791 tests. The test run completed in 39.25 seconds. Six test files failed with a total of 73 individual test failures. Fourteen tests are skipped. The remaining 1,704 tests pass. These numbers represent the floor that the v1.0 implementation must not regress below. Additionally, `tsc --noEmit` passes with zero type errors, and `next build` compiles successfully in 49 seconds with all 180 static pages generated.

2.1 Test Summary

Metric	Value
Test Files Total	48
Test Files Passed	42
Test Files Failed	6
Total Tests	1,791
Tests Passed	1,704
Tests Failed	73
Tests Skipped	14
Pass Rate	95.1%
Duration	39.25s
TypeScript (tsc --noEmit)	PASS (zero errors)
Next.js Build	PASS (compiled in 49s, 180 pages)

2.2 Failing Test Files

The six failing test files cluster into two distinct root causes. The first cluster involves the Intelligence API middleware and envelope contract tests (tickets 1, 2, and deep coverage), where the `VALID_INCLUDES` set has a count mismatch, envelope responses are not consistently structured, and the `intelligenceGuard` middleware does not reject invalid include parameters as expected. The second cluster involves the Account Prioritization Engine tests (ticket 4), where `parseRevenueBreakdown` is not exported from the engine module, causing all score unification and integration tests to fail with `TypeError: parseRevenueBreakdown is not a function`. Additionally, the `getICPProfile()` helper does not correctly parse JSON values from SystemSetting records, leading to cascading failures in static fit scoring.

Test File	Failed	Root Cause
`tests/ticket-deep-coverage.test.ts`	3	VALID_INCLUDES count mismatch (22 vs actual); computeFreshness stale logic; intelligenceGuard reject behavior
`tests/ticket1-intelligence-validation.test.ts`	7	Schema validation: includeSchema rejects invalid values not enforced; companyIdSchema rejects empty string not enforced
`tests/ticket1-intelligence-integration.test.ts`	1	Non-existent company returns wrong error format
`tests/ticket2-integration.test.ts`	16	Intelligence API envelope contract: 10 endpoints missing utilitySuccess wrapper; freshness not in meta; include selective loading broken
`src/lib/account-prioritization/__tests__/engine.test.ts`	21	parseRevenueBreakdown not exported; getICPProfile JSON parse error; scoreStaticFit edge cases; scoreTimingUrgency signal recency
`src/lib/account-prioritization/__tests__/ticket4-score-unification.test.ts`	25	Same parseRevenueBreakdown missing export; normalizeRevenueCategory mapping wrong; tier classification boundary values; null handling

2.3 Failure Breakdown by Root Cause

Root Cause Category	Tests Affected	Files	Severity
parseRevenueBreakdown not exported	46	2 (engine, ticket4)	P1 - Blocking
Intelligence API envelope inconsistency	16	1 (ticket2)	P1 - Contract violation
VALID_INCLUDES count mismatch	3	1 (deep-coverage)	P2 - Validation gap
getICPProfile JSON parse error	21	2 (engine, ticket4)	P1 - Scoring broken
Schema validation not enforced	7	1 (ticket1-validation)	P2 - Input validation gap
Tier boundary classification	8	1 (ticket4)	P2 - Business logic

3. Runtime Baseline

The runtime baseline documents the operational state of the DeepMindQ platform at checkpoint time. The database is a local SQLite file at `db/custom.db` (2.0 MB) with 72 tables created by Prisma but containing minimal data. The environment is configured with a single `.env` variable (`DATABASE_URL`). No registered users exist in the system, no active sessions, and no seed data beyond what appears to be development test records. The authentication system is technically functional (login, OTP, password reset routes exist) but operates in an empty-user environment.

3.1 Database State

Metric	Value
Database Engine	SQLite 3.x (version 3.46.0)
Database File	db/custom.db (2.0 MB)
Total Tables	72
Prisma Migrations	1 (20260724_wave8a_intelligence_object.sql)
Registered Users	0
Active Sessions	0
OtpCode Records	0
Total Data Rows (all tables)	87

3.2 Tables With Data

Of the 72 database tables, only 15 contain any data. The largest tables are CompanySignal (43 rows) and CompanyNote (10 rows), which appear to be development test data. Most business-critical tables (User, Session, AIGenerationAudit, EmailSequence, Playbook, KnowledgeEntry, Embedding) are completely empty. This confirms the platform has never been seeded with production or gold-standard data.

Table	Rows	Likely Source
CompanySignal	43	Dev test data — AI-generated signal records
CompanyNote	10	Dev test data — manual company notes
Contact	10	Dev test data — sample contact records
ImportBatch	9	Dev test data — data import history
Company	3	Dev test data — sample company records
Reply	10	Dev test data — email reply records
CompanyTimelineEvent	2	Dev test data — timeline events
ContactNote	3	Dev test data — contact notes
OpportunityRecommendation	2	Dev test data — AI recommendations
AccountStrategy	1	Dev test data — account strategy
CapabilityAsset	1	Dev test data — capability library
CompanyResearchCard	1	Dev test data — research card

Table	Rows	Likely Source
SignalCapabilityMatch	1	Dev test data — signal matching
SystemSetting	1	user_preferences = {"tone":"formal"}
All Other Tables (57)	0	Completely empty

3.3 Environment Configuration

The environment configuration is minimal. The `.env` file contains exactly one variable: the database connection URL. No external service credentials (API keys for LLM providers, email service, etc.) are configured. The platform relies on the ZAI SDK for LLM operations, which is available in the deployment environment but requires no local API key configuration. This minimal configuration confirms the platform is in a pre-production state with no production secrets or external integrations configured.

Variable	Value	Notes
DATABASE_URL	file:/home/z/my-project/db/custom.db	Local SQLite — no remote DB
LLM Provider Keys	Not configured	Relies on ZAI SDK runtime injection
Session Secret	Not configured	No JWT_SECRET or session secret in .env
Email Service	Not configured	No SMTP or email API credentials
Redis / Cache	Not configured	No external cache layer

3.4 Authentication Flow

The authentication system has a complete surface of routes (login, register, logout, OTP verification, password reset, session management via `/api/auth/me`). However, the actual security posture is severely compromised by two critical issues. First, the proxy middleware at `src/proxy.ts:95` only checks for the `dmq_session` cookie presence without validating it against the database. Any client that sets a `dmq_session` cookie (even a fabricated one) will pass the proxy and reach all API routes. Second, the `/api/auth/me` endpoint at line 38-52 contains a catch block that, upon any database failure, returns a hardcoded admin identity (`shanker001@gmail.com` with role `admin`). This means any error condition in the authentication flow grants full admin access. Combined with the fact that no API routes (except one) import any authentication middleware, the platform is effectively open access.

Component	File	Status	Risk
Proxy Cookie Check	`src/proxy.ts:95`	Presence-only check	P0 — Fabricated cookie bypasses all auth
Auth /me Fallback	`src/app/api/auth/me/route.ts:38-52`	Hardcoded admin on DB error	P0 — Error grants admin access
checkApiAuth()	`src/lib/api-auth.ts`	Exported but never imported	P1 — Dead code, 0 routes protected
requireAdminRole()	`src/lib/api-auth.ts`	Exported but never imported	P1 — Dead code, no admin enforcement

Component	File	Status	Risk
Seed Routes	`src/app/api/seed/route.ts`	No authentication	P0 — Destructive DB ops publicly accessible
Export Route	`src/app/api/export/route.ts`	No auth, no limits	P0 — Mass data exfiltration

4. Critical Findings Reference (Pre-Existing)

These findings were documented in the previous Reality Verification Matrix and remain unresolved at this checkpoint. They are included here as a reference so that progress can be tracked against the v1.0 remediation plan. Each finding has been independently verified with direct file:line evidence and confirmed as genuine, not false positives. None of the 15 findings from the verification audit have been addressed.

ID	Severity	Finding	File	Status
F01	P0	Proxy auth: cookie presence only, no DB validation	`src/proxy.ts:95`	Open
F02	P0	/auth/me hardcoded admin fallback on DB error	`src/app/api/auth/me/route.ts:38-52`	Open
F03	P0	222/223 API routes without authentication	`src/app/api/**`	Open
F04	P0	Seed routes: destructive DB ops, zero auth	`src/app/api/seed/route.ts`	Open
F05	P0	Export route: no auth, no row limits, no audit	`src/app/api/export/route.ts`	Open
F06	P1	Reasoning engine risk filter always true (operator precedence)	`src/lib/enterprise-reasoning-engine.ts:250`	Open
F07	P1	LLM client system prompt role wrong (assistant vs system)	`src/lib/llm-client.ts:145`	Open
F08	P1	AI token/cost tracking hardcoded to 0	`src/lib/llm-client.ts:584-588`	Open
F09	P1	usage-tracker writes to wrong table (AIGenerationAudit not AIUsageLog)	`src/lib/ai-copilot/usage-tracker.ts:87-99`	Open
F10	P1	Intelligence API VALID_INCLUDES count mismatch	`src/lib/intelligence-api/middleware.ts:64`	Open
F11	P1	Intelligence API envelope inconsistency (5/6 endpoints)	`src/app/api/intelligence/**`	Open
F12	P1	parseRevenueBreakdown not exported from engine	`src/lib/account-prioritization/engine.ts`	Open
F13	P2	requireAdminRole defined but never imported	`src/lib/api-auth.ts:47-55`	Open

ID	Severity	Finding	File	Status
F14	P2	/api/seed has no auth guard (documented as non-public)	`src/lib/auth-helpers.ts:19`	Open
F15	P2	AIUsageLog model exists but never written to	`prisma/schema.prisma:2785`	Open

5. Production Readiness Scores (Pre-Fix)

The following scores assess the platform across six critical dimensions, each scored on a 1-10 scale where 10 represents full production readiness. These scores are based on the evidence gathered in this checkpoint and the 15 verified findings from the Reality Verification Matrix. They represent the starting line for the v1.0 remediation effort. The composite score of 2.8 out of 10 reflects a platform that compiles and builds successfully but has critical security gaps, broken business logic, and insufficient test coverage for its claimed-completed features.

5.1 Scoring Matrix

Dimension	Score	Rating	Justification
Security	1	Critical	P0: Proxy checks cookie presence only. /auth/me returns hardcoded admin on error. 222/223 routes unprotected. Seed and export routes have zero auth.
Reliability	4	Marginal	Build and TSC pass. 73 tests fail across 6 files. Reasoning engine filter bug lets all signals through. Auth fallback masks failures.
AI Quality	2	Critical	LLM system prompt role is wrong (assistant vs system). Token/cost tracking hardcoded to 0. Usage tracker writes to wrong table. Reasoning engine filter broken.
Data Integrity	3	Poor	Schema is sound (96 models). But AIUsageLog never written to. Export route has no limits. Seed routes can destroy data. No row-level security.
Testing	5	Marginal	1,704/1,791 tests pass (95.1%). But failures expose real bugs: missing exports, wrong envelope contracts, validation gaps. No E2E or integration tests for auth.
Deployment Readiness	2	Critical	No staging environment. No migration strategy beyond one file. No environment variables for production. No health checks for critical services. No rollback plan.

Composite Score	Rating	Assessment
2.8 / 10	NOT PRODUCTION READY	Platform compiles and serves pages but has critical security vulnerabilities, broken business logic in AI engines, unreliable authentication, and no operational safeguards. Requires significant remediation before any production deployment.

6. Production Foundation v1.0 Implementation Rules

The following rules are binding for the entire v1.0 implementation effort. They are designed to prevent the most common failure patterns observed in the current codebase: masking bugs with permissive tests, introducing regressions in unrelated subsystems, and making uncoordinated changes to shared contracts. Every implementation decision must be evaluated against these rules before, during, and after code changes.

6.1 Test Modification Rules

Primary Rule: Never modify tests just to make them pass. The default action when a test fails is to fix the production code that causes the failure. Only modify a test when you can prove with evidence from the ARCHITECTURE.md specification or documented business requirements that the test expectation is incorrect. Every test change requires a documented justification explaining: (1) why the original test was wrong, (2) the evidence from code or specification that proves it, and (3) why the new expectation is correct.

Required documentation for every test change:

1. **Why the test was wrong:** Specific reference to the ARCHITECTURE.md section or business requirement that the original test violated.
2. **Code/spec evidence:** The exact file:line in ARCHITECTURE.md or the code contract that proves the original expectation was incorrect.
3. **Why the new expectation is correct:** How the updated test aligns with the documented specification and does not simply mask a production bug.

6.2 Regression Prevention

Every code change must be validated against the frozen baseline recorded in this document. The following gates must pass before any change is committed:

1. All 1,704 currently passing tests must continue to pass. Any regression in a previously passing test blocks the change.
2. TypeScript compilation must remain at zero errors (`tsc --noEmit`).
3. The Next.js build must continue to succeed without errors.
4. No new P0 or P1 findings may be introduced by any change.

6.3 Change Scope Rules

Changes must be scoped to the specific ticket or fix they address. Cross-cutting changes (especially to shared middleware, Prisma schema, or response envelope patterns) require explicit impact analysis documented in the worklog before implementation. Changes to the proxy middleware (`src/proxy.ts`) affect all 223 API routes simultaneously and must be treated as high-risk changes requiring full test suite validation.

6.4 Authentication Remediation Order

Given that 222 routes currently lack authentication, enforcing auth must be done in a staged approach to prevent breaking the entire platform. The recommended order is: (1) fix the proxy middleware to validate sessions against the database, (2) add public route whitelist for login, register, health, and static assets, (3) import `checkApiAuth` into high-risk routes (seed, export, admin, user management), (4) progressively add auth to remaining routes by domain area, (5) add `requireAdminRole` to admin-only endpoints. Each stage must be a separate commit with its own test validation.

7. Known Critical Files

These files contain verified bugs, security vulnerabilities, or business logic errors that must be addressed during the v1.0 remediation. They are listed with their line counts to indicate the scope of changes needed. The total is 2,074 lines across 6 files, representing the core of the remediation effort.

File	Lines	Critical Issue
`src/lib/enterprise-reasoning-engine.ts`	667	Line 250: Risk filter always true (operator precedence bug). All risk signals pass through unfiltered.
`src/lib/llm-client.ts`	594	Line 145: System prompt role wrong. Lines 584-588: Token/cost tracking hardcoded to 0.
`src/lib/intelligence-api/middleware.ts`	321	Line 64: VALID_INCLUDES count mismatch. Inconsistent createResponse() usage.
`src/lib/ai-copilot/usage-tracker.ts`	214	Lines 87-99: Writes to AIGenerationAudit instead of AIUsageLog. Line 128: Queries wrong table.
`src/proxy.ts`	221	Line 95: Cookie presence-only check, no DB validation. Single point of auth failure.
`src/app/api/auth/me/route.ts`	57	Lines 38-52: Hardcoded admin identity on database error. P0 security hole.